

Sampling Distributions and Plug-In Estimators

Math 742

1/30/2026

Motivation: Why Sampling Distributions Matter

A statistic is a function of random data $\rightarrow$ it is itself a random variable.

Any estimator has:

- a distribution (sampling distribution),
- a center (bias),
- a spread (variance),
- a shape.

Real goal: quantify how much variability is induced by sampling. This variability is exactly what drives confidence intervals, hypothesis tests, and estimator evaluation.

Leads naturally to plug-in estimation (estimating functionals of the distribution).

Sampling Distributions

Definition 1: Let $X_1, X_2, \dots, X_n$ be an iid sample from a population whose density is $f(\mathbf{x}|\theta)$. The statistic T computed from the sample has its own distribution induced by repeated sampling from $f(x|\theta)$. This distribution is called the **sampling distribution**.

Example: If $X_i \sim N(\mu, \sigma^2)$, then $\bar{X} = \frac{X_1 + X_2 + \dots + X_n}{n} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$.

Example: If $\hat{p} = \frac{1}{n} \sum_{i=1}^n X_i$, then $\hat{p} \sim \text{Binomial}(n, p)/n$.

Theorem: Let $X_1, X_2, \dots, X_n$ be an iid sample from a population whose mean is μ and variance $\sigma^2 < \infty$, then:

$$(1) \mathbb{E}(\bar{X}) = \mu$$

$$(2) \mathbb{V}(\bar{X}) = \frac{\sigma^2}{n}$$

$$(3) \mathbb{E}(S^2) = \sigma^2$$

One quick note on notation. Generally, Latin letters represent values from the sample (e.g. $\bar{X}$ and S^2) and Greek letters indicate values from the population (e.g. σ^2). Furthermore, “capital” letters represent random variables and lower case letters represent a realization of that random variable (i.e. a specific value).

Proof: (1) $\mathbb{E}(\bar{X}) = \mu$.

Since $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ and expectation is linear,

$$\mathbb{E}(\bar{X}) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}(X_i) = \frac{1}{n} \cdot n\mu = \mu.$$

(2) $\mathbb{V}(\bar{X}) = \frac{\sigma^2}{n}$.

Using independence and the fact that $\mathbb{V}(aY) = a^2\mathbb{V}(Y)$,

$$\mathbb{V}(\bar{X}) = \mathbb{V}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \mathbb{V}(X_i) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}.$$

(3) $\mathbb{E}(S^2) = \sigma^2$.

We use the useful identity

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

Taking expectations gives,

$$\begin{aligned} \mathbb{E}\left[\sum_{i=1}^n (X_i - \bar{X})^2\right] &= \sum_{i=1}^n \mathbb{E}[(X_i - \mu)^2] - n \mathbb{E}[(\bar{X} - \mu)^2] \\ &= n\sigma^2 - n \frac{\sigma^2}{n} \\ &= (n-1)\sigma^2 \end{aligned}$$

So, we have shown:

$$\begin{aligned} \mathbb{E}\left[\sum_{i=1}^n (X_i - \bar{X})^2\right] &= (n-1)\sigma^2 \\ \implies \mathbb{E}\left[\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n-1}\right] &= \sigma^2 \end{aligned}$$

or, $\mathbb{E}(S^2) = \sigma^2$

Standard Methods to Derive Sampling Distributions

- Method A: Linear transformations. We used this above to show that

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

when $X_i \sim N(\mu, \sigma^2)$.

This follows the fundamental fact:

Any linear combination of jointly normal random variables is also normally distributed. The mean follows from linearity of expectation, and the variance comes from the usual variance-covariance rules.

In this case,

$$\bar{X} = \frac{1}{n}(X_1 + X_2 + \dots + X_n)$$

is a linear combination with coefficients $(1/n)$.

Thus,

$$\mathbb{E}(\bar{X}) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}(X_i) = \mu$$

$$\mathbb{V}(\bar{X}) = \frac{1}{n^2} \sum_{i=1}^n \mathbb{V}(X_i) = \frac{\sigma^2}{n}$$

Just a linear transformation of normals.

- Method B: Jacobian transformation

Example: Let X_1, X_2 be two independent samples from $\text{Exp}(\lambda)$.

Use the Jacobian transformation method to find the distribution of T :

$$T = X_1 + X_2$$

Define a transformation:

$$T = X_1 + X_2, \quad W = \frac{X_1}{X_1 + X_2}.$$

Notice that W is the proportion of the sum contributed to by X_1 .

Using this transformation we see that $X_1 = WT$ and $X_2 = (1 - W)T$.

Note: $X_1 \geq 0$ and $X_2 \geq 0$. Since $X_1 = WT \implies W \geq 0$. Also, $X_2 = (1 - W)T \geq 0 \implies W \leq 1$ ($0 \leq W \leq 1$). It's the transformation that determines our limits, not the Jacobian.

The Jacobian of the transformation $(X_1, X_2) \mapsto (T, W)$ is:

$$J = \begin{vmatrix} \frac{\partial X_1}{\partial T} & \frac{\partial X_1}{\partial W} \\ \frac{\partial X_2}{\partial T} & \frac{\partial X_2}{\partial W} \end{vmatrix} = \begin{vmatrix} W & T \\ 1 - W & -T \end{vmatrix} = -WT - (1 - W)T = -WT - T + WT = -T$$

$$|J| = T$$

(Note: $\det(J) = -T, |\det(J)| = T$)

Original joint pdf of (X_1, X_2) , because we have independence, we multiply.

$$f_{X_1, X_2}(x_1, x_2) = \lambda^2 e^{-\lambda x_1} e^{-\lambda x_2} = \lambda^2 e^{-\lambda(x_1 + x_2)}$$

$$f_{T, W}(t, w) = f_{X_1, X_2}(x_1 = wt, x_2 = (1 - w)t) \cdot |J| = \lambda^2 e^{-\lambda t}, \quad 0 < w < 1, t > 0$$

Now we find the marginal distribution of T by integrating out W .

First notice that When finding the marginal distribution of T we have limits of $W = 0$ and $W = 1$.

$$f_T(t) = \int_0^1 f_{T, W}(t, w) dw = \lambda^2 t e^{-\lambda t} w|_0^1 = \lambda^2 t e^{-\lambda t}, \quad t > 0$$

We could've done this problem with the moment generating function.

- Method C: Method of moments

Definition: For each integer n , the n^{th} moment of X (i.e. μ'_n) is equal to $\mathbb{E}(X^n)$. The n^{th} central moment is $\mu_n = \mathbb{E}[(X - \mu)^n]$.

Definition: Let X be a random variable with CDF F_X . The **moment generating function (MGF)** of X is denoted by $M_X(t) = \mathbb{E}[e^{tX}]$.

Let $X_1, \dots, X_n$ be independent random variables with $X_i \sim \text{Exp}(\lambda)$ (rate parameter $\lambda > 0$). And, $T = \sum_{i=1}^n X_i$. Then $T \sim \Gamma(n, \lambda)$ (shape = n , rate = λ).

The MGF of an $\text{Exp}(\lambda)$ random variable X is, for $t < \lambda$,

$$\begin{aligned} M_X(t) &= \mathbb{E}[e^{tX}] = \int_0^\infty e^{tx} \lambda e^{-\lambda x} dx \\ &= \lambda \int_0^\infty e^{-(\lambda-t)x} dx = \lambda \cdot \frac{1}{\lambda-t} = \frac{\lambda}{\lambda-t}, \quad t < \lambda. \end{aligned}$$

Because the X_i are independent, $e^{tX_1}, e^{tX_2}, \dots, e^{tX_n}$ are also independent. Therefore,

$$\begin{aligned} M_T(t) &= \mathbb{E}\left[e^{t \sum_{i=1}^n X_i}\right] = \mathbb{E}\left[\prod_{i=1}^n e^{tX_i}\right] \\ &= \prod_{i=1}^n \mathbb{E}(e^{tX_i}) = \prod_{i=1}^n M_{X_i}(t) \end{aligned}$$

Therefore,

$$M_T(t) = \left(\frac{\lambda}{\lambda-t}\right)^n, \quad t < \lambda.$$

But the MGF of a $\Gamma(\alpha, \lambda)$ random variable with shape α and rate λ is

$$M_{\Gamma(\alpha, \lambda)}(t) = \left(\frac{\lambda}{\lambda-t}\right)^\alpha, \quad t < \lambda.$$

Setting $\alpha = n$ shows that $M_T(t) = M_{\Gamma(n, \lambda)}(t)$ for $t < \lambda$.

Since the MGF uniquely determines the distribution on an open interval containing 0, we conclude that

$$T = \sum_{i=1}^n X_i \sim \Gamma(n, \lambda).$$

We now verify this result empirically via simulation.

Simulation:

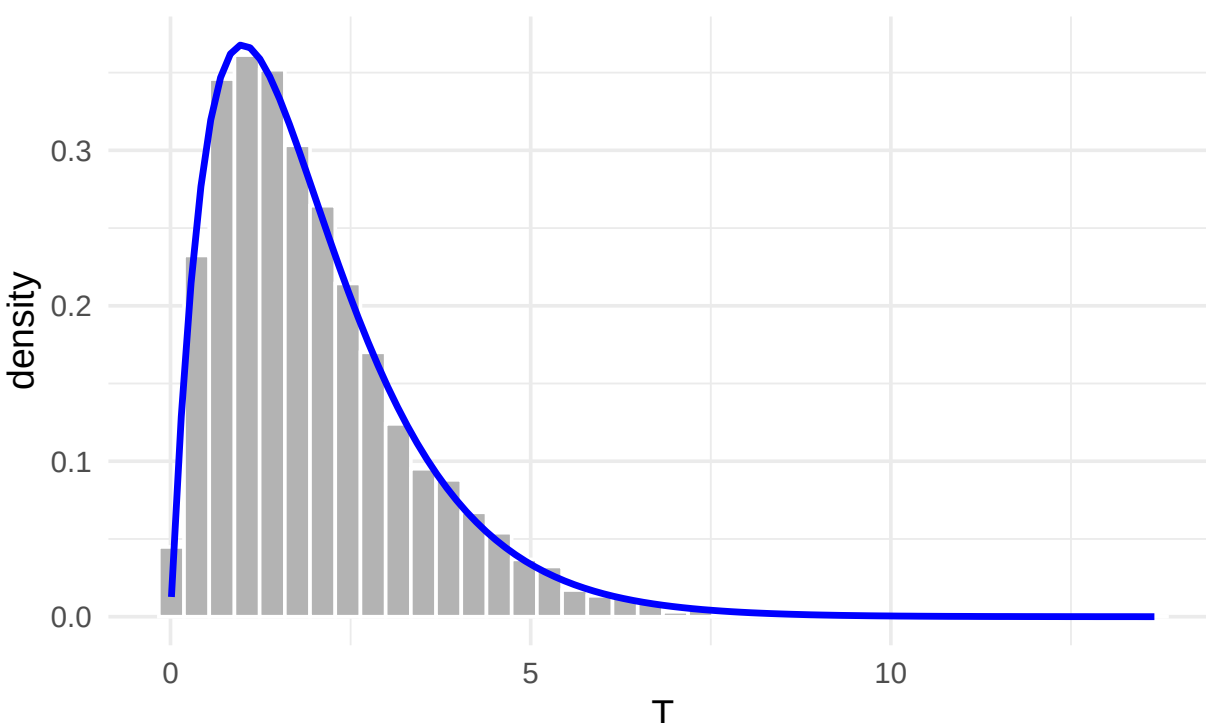
```
set.seed(123)
lambda <- 1
Nsim <- 10000

# Simulate X1 and X2
X1 <- rexp(Nsim, rate = lambda)
X2 <- rexp(Nsim, rate = lambda)
T <- X1 + X2
```

```
# Histogram with theoretical Gamma overlay
library(ggplot2)
df <- data.frame(T = T)

ggplot(df, aes(T)) +
  geom_histogram(aes(y=after_stat(density)), bins = 40, fill = "grey70", color = "white") +
  stat_function(fun = function(t) dgamma(t, shape = 2, rate = lambda),
               color = "blue", linewidth = 1.2) +
  ggtitle("Histogram of T = X1 + X2 with \n Gamma(2, lambda) Overlay") +
  theme_minimal(base_size = 14)
```

Histogram of T = X1 + X2 with
Gamma(2, lambda) Overlay



The match between empirical and theoretical distributions helps verify our derivation.

- Method D: Recognizing known forms (e.g. sum of the squares of normal random variables is χ^2)

If $Z_1, Z_2, \dots, Z_k \sim N(0, 1)$, the $\sum_{i=1}^n Z_i^2 \sim \chi_k^2$

Why Sampling Distributions Matter

- (1) Confidence intervals

Confidence interval for the mean. Because,

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

$$\mathbb{P}\left(\bar{X} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}\right)$$

Note: This assumes σ is known. When it is unknown, we have another sampling distribution, the t-distribution.

$$\mathbb{P}\left(\bar{X} - t_{\alpha/2, n-1} \frac{s}{\sqrt{n}} \leq \mu \leq \bar{X} + t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}\right)$$

(2) Hypothesis tests: Reject $H_0 : \mu = \mu_0$ if

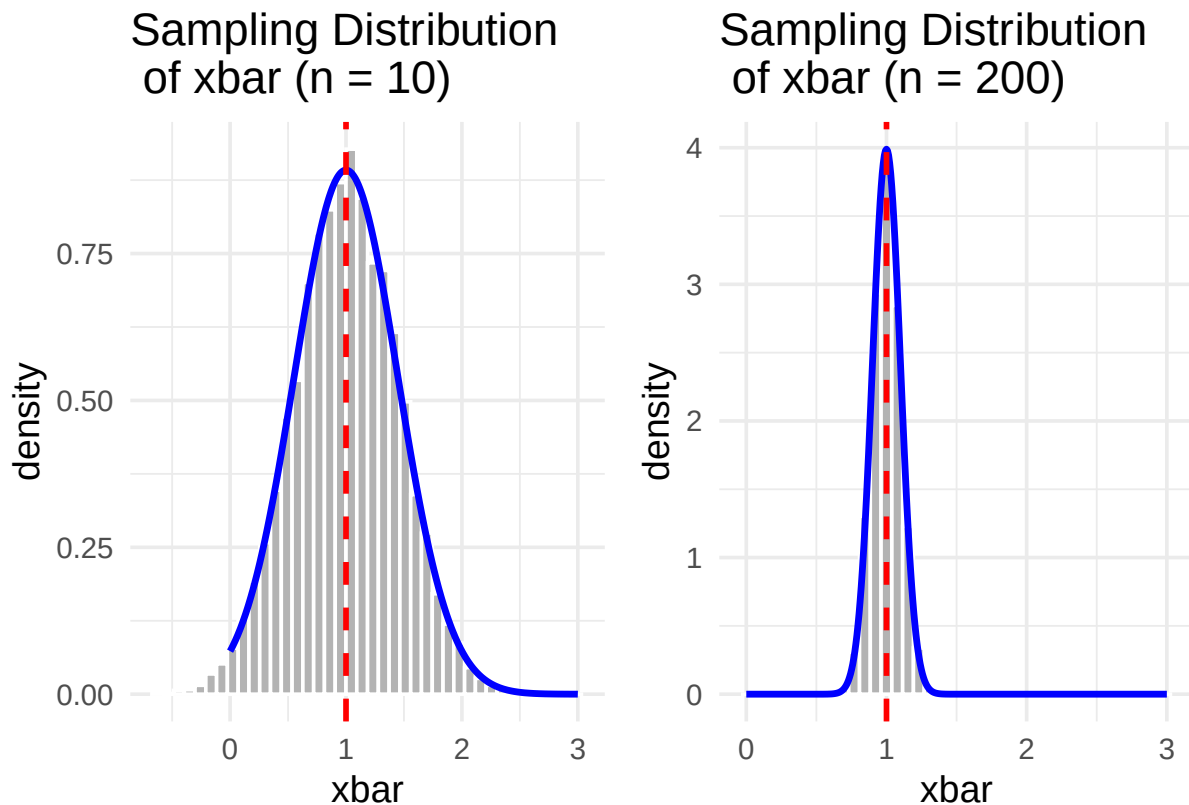
$$\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} > z_{0.95} ; \sigma \text{ known}$$

or,

$$\frac{\bar{X} - \mu_0}{s/\sqrt{n}} > t_{0.95, n-1} ; \sigma \text{ unknown}$$

(3) Understanding estimator variability

→ Sampling from a normal population

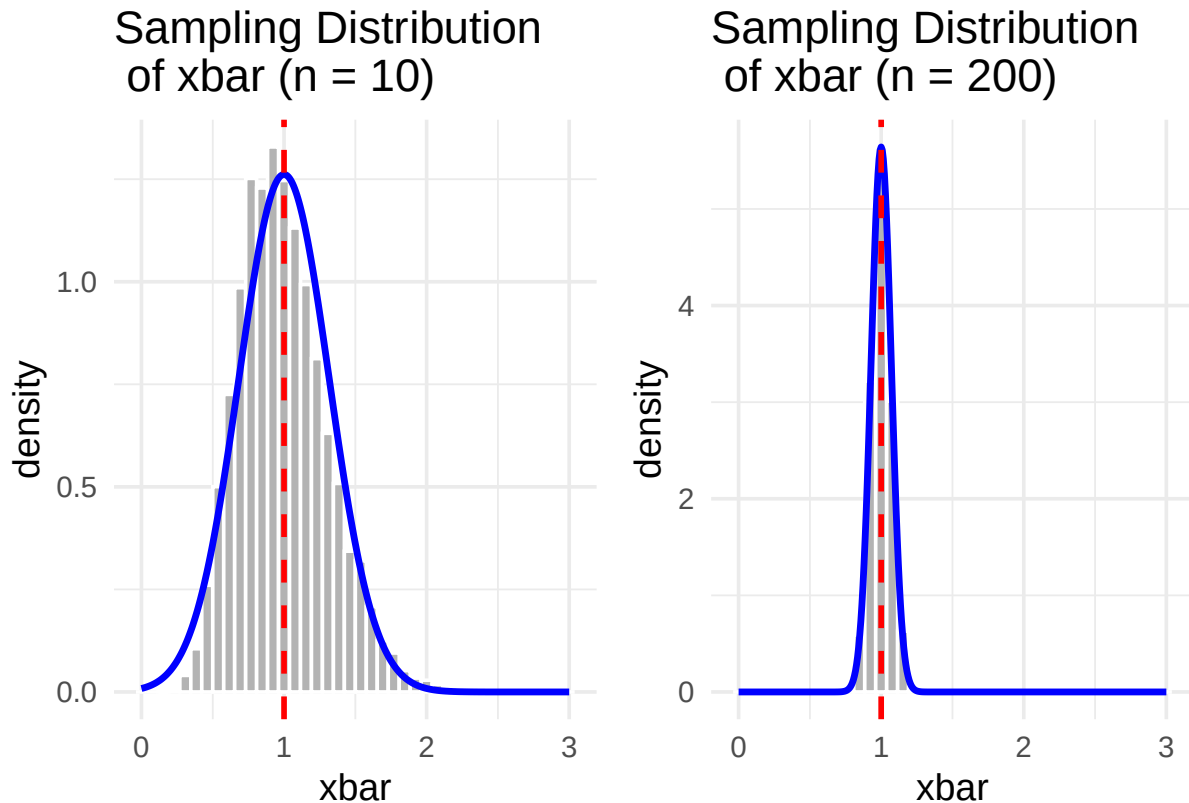


Left panel ($n = 10$). Histogram is wide, quite good normal approximation.

Right panel ($n = 100$). Histogram is narrow, almost perfectly normal.

(4) Large sample approximations via the Central Limit Theorem

→ Sampling from an Exponential Distribution



Left panel ($n = 10$). Histogram is wide, fairly decent normal approximation.

Right panel ($n = 100$). Histogram is narrow, almost perfectly normal.

Plug-In Estimators: The Big Idea

What Is a Plug-In Estimator?

For a parameter-dependent quantity $g(\theta)$, the plug-in estimator is:

$$\widehat{g(\theta)} = g(\hat{\theta})$$

where $\hat{\theta}$ is an estimator of θ .

→ Plug-in estimators trade simplicity for bias control.

Here is one way to think about a plug-in estimator. Suppose you have a weather model that needs the true humidity (θ) in order to compute the chance of rain.

$$g(\theta) = \text{chance of rain}$$

There is no way to measure the humidity exactly, your instrument has error with it, so it only gives you an estimate of the humidity ($\hat{\theta}$). Because this is all you have, you run your model with an estimate of the humidity → $g(\hat{\theta})$. You plug-in your estimate of θ , compute the chance of rain and go with that.

Why the Plug-In Principle Is Universal

- Once you estimate θ , you estimate anything else by plugging it in.
- This is often the default method unless a special estimator is constructed.
- MLE (covered later) helps because it satisfies:

$$g(\hat{\theta}_{MLE}) = \widehat{g(\theta)}_{MLE}$$

Examples of Plug-In Estimators

(1) Variance of a Bernoulli

- The true variance is $p(1-p)$
- The plug-in estimator using $\hat{p} = \bar{X}$:

$$\widehat{\mathbb{V}(X)} = \hat{p}(1 - \hat{p})$$

Because,

$$\mathbb{E}[\hat{p}(1 - \hat{p})] = \frac{n-1}{n}p(1-p)$$

This shows the plug-in estimator is biased downward, since $(n-1)/n < 1$.

(2) Normal Distribution: Estimating Functionals

We want to estimate σ^2, μ^2 and $\mathbb{P}(X > c) = 1 - \Phi\left(\frac{c-\mu}{\sigma}\right)$

Plug-in versions:

$$\hat{\sigma}^2, \hat{\mu}^2, 1 - \Phi\left(\frac{c - \hat{\mu}}{\hat{\sigma}}\right)$$

Properties of Plug-In Estimators

(1) If $\hat{\theta}$ is unbiased, $g(\hat{\theta})$ doesn't need to be.

Example: Let $X_1, X_2, \dots, X_n \sim N(\theta, 1)$, where $\hat{\theta} = \bar{X}$.

$$\mathbb{E}(\hat{\theta}) = \mathbb{E}\left(\frac{X_1 + X_2 + \dots + X_n}{n}\right) = \frac{1}{n}\mathbb{E}(X_1 + X_2 + \dots + X_n) = \frac{1}{n}n\mu = \mu$$

So, $\hat{\theta}$ is unbiased. Now, I show that $\hat{\theta}^2$ is biased.

$$\mathbb{V}(\hat{\theta}) = \mathbb{E}(\hat{\theta}^2) - [\mathbb{E}(\hat{\theta})]^2$$

So,

$$\begin{aligned}\mathbb{V}(\bar{X}) &= \mathbb{E}(\bar{X}^2) - [\mathbb{E}(\bar{X})]^2 \\ \mathbb{E}(\bar{X}^2) &= \mathbb{V}(\bar{X}) + [\mathbb{E}(\bar{X})]^2\end{aligned}$$

Note: $\mathbb{E}(\bar{X}) = \theta$, and since $\bar{X} \sim N(\theta, 1/n)$ we have $\mathbb{V}(\bar{X}) = 1/n$.

$$\mathbb{E}(\hat{\theta}^2) = \mathbb{E}(\bar{X}^2) = \frac{1}{n} + \theta^2 \neq \hat{\theta}^2$$

Unbiasedness does not survive nonlinear transformations!
--

Example: If we do a straight average, σ^2 is biased. If we divide by $n - 1$ it is unbiased.

Let $X_i \sim N(\mu, \sigma^2)$,

$$\hat{\sigma}_{\text{plug-in}}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$$

```
set.seed(123)

# True parameters
mu <- 5
sigma <- 2

n <- 10
Nsim <- 10000

plug_in_var <- replicate(Nsim, {
  x <- rnorm(n, mu, sigma)
  sum((x - mean(x))^2)/n      # divisor n
})

# Compare empirical mean with true sigma^2 = 4
mean(plug_in_var)

## [1] 3.60371

true_sigma2 <- sigma^2
true_sigma2
```

```
## [1] 4
```

Biased, The actual variance is 4 and our estimate is 3.6.

Now, divide by $n - 1$ instead,

```
set.seed(123)

# True parameters
mu <- 5
sigma <- 2

n <- 10
Nsim <- 10000

plug_in_var <- replicate(Nsim, {
  x <- rnorm(n, mu, sigma)
  sum((x - mean(x))^2)/(n-1)    # divisor n-1
})

# Compare empirical mean with true sigma^2 = 4
mean(plug_in_var)

## [1] 4.004122
```

```
true_sigma2 <- sigma^2
true_sigma2
```

```
## [1] 4
```

Summary / Takeaways

- A statistic is a random variable; its distribution is fundamental.
- Plug-in estimation is the simplest, most widely used method for estimating functionals.
- Most estimators in practice are plug-in estimators.
- Understanding sampling distributions is essential for assessing their quality.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gamma

# Set seed and parameters
np.random.seed(123)
lambda_param = 1
Nsim = 10000

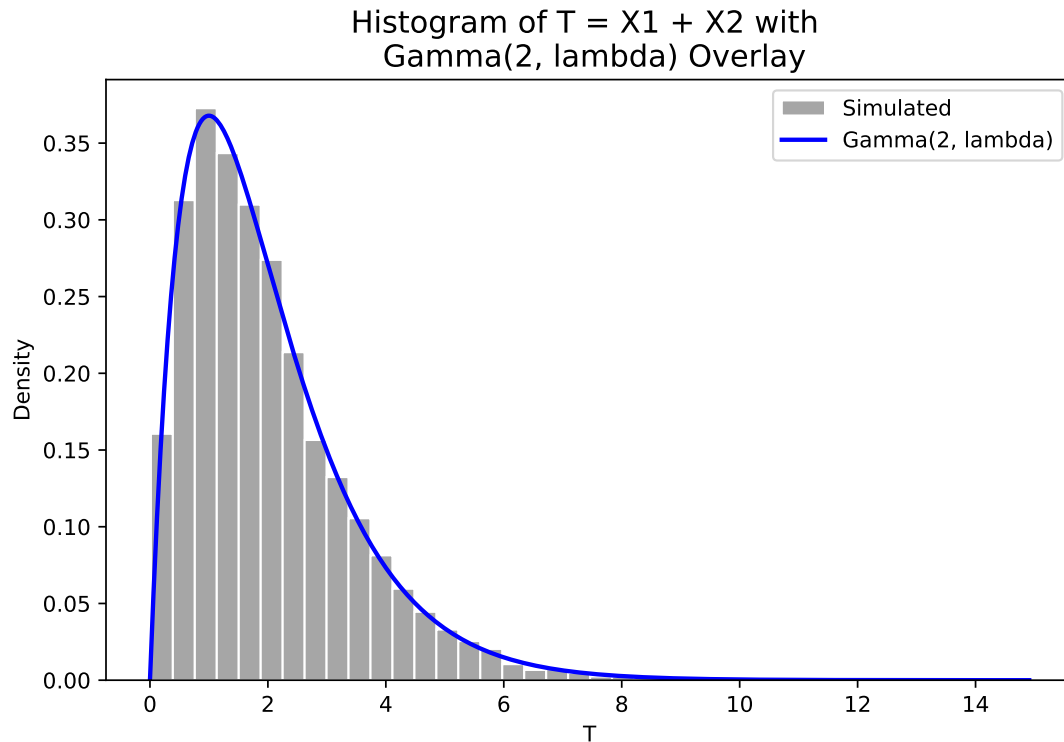
# Simulate X1 and X2 ~ Exponential(lambda)
X1 = np.random.exponential(1 / lambda_param, Nsim)
X2 = np.random.exponential(1 / lambda_param, Nsim)
T = X1 + X2

# Plot histogram with theoretical Gamma(2, lambda) overlay
plt.figure(figsize=(8,5))

# Histogram
plt.hist(T, bins=40, density=True, color='grey', edgecolor='white', alpha=0.7, label='Simulated')

# Theoretical Gamma PDF (shape=2, scale=1/lambda)
t_values = np.linspace(0, max(T), 500)
plt.plot(t_values, gamma.pdf(t_values, a=2, scale=1/lambda_param),
         color='blue', linewidth=2, label='Gamma(2, lambda)')

# Titles and labels
plt.title("Histogram of T = X1 + X2 with \n Gamma(2, lambda) Overlay", fontsize=14)
plt.xlabel("T")
plt.ylabel("Density")
plt.legend()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Set seed
np.random.seed(123)

# Number of simulations
Nsim = 10000

# True distribution parameters
true_mean = 1
true_var = 2

# Sample means
xbar_10 = [np.mean(np.random.normal(true_mean, np.sqrt(true_var), 10)) for _ in range(Nsim)]
xbar_200 = [np.mean(np.random.normal(true_mean, np.sqrt(true_var), 200)) for _ in range(Nsim)]

# Theoretical normal densities for CLT comparison
x_vals = np.linspace(0, 3, 400)
d10 = norm.pdf(x_vals, loc=true_mean, scale=np.sqrt(true_var / 10))
d200 = norm.pdf(x_vals, loc=true_mean, scale=np.sqrt(true_var / 200))

# Create side-by-side plots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Plot for n = 10
axes[0].hist(xbar_10, bins=40, density=True, color='grey', edgecolor='white', alpha=0.7)
axes[0].plot(x_vals, d10, color='blue', linewidth=2, label='Theoretical PDF')
```

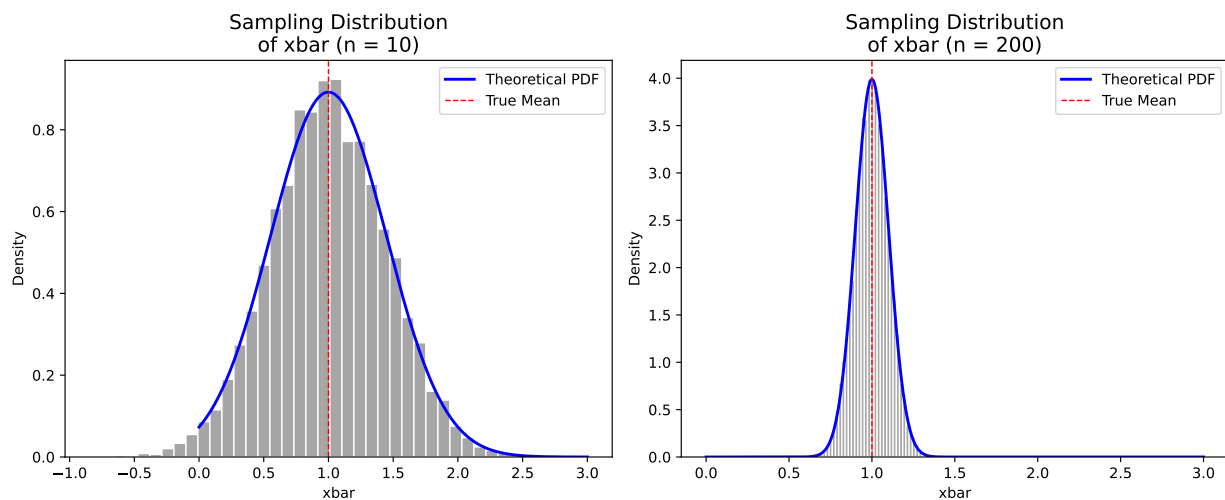
```

axes[0].axvline(true_mean, color='red', linestyle='--', linewidth=1, label='True Mean')
axes[0].set_title('Sampling Distribution\nof xbar (n = 10)', fontsize=14)
axes[0].set_xlabel('xbar')
axes[0].set_ylabel('Density')
axes[0].legend()

# Plot for n = 200
axes[1].hist(xbar_200, bins=40, density=True, color='grey', edgecolor='white', alpha=0.7)
axes[1].plot(x_vals, d200, color='blue', linewidth=2, label='Theoretical PDF')
axes[1].axvline(true_mean, color='red', linestyle='--', linewidth=1, label='True Mean')
axes[1].set_title('Sampling Distribution\nof xbar (n = 200)', fontsize=14)
axes[1].set_xlabel('xbar')
axes[1].set_ylabel('Density')
axes[1].legend()

plt.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Set seed
np.random.seed(123)

# Number of simulations
Nsim = 10000

# True distribution parameters
lambda_param = 1
true_mean = 1 / lambda_param
true_var = 1 / lambda_param**2

# Sample means
xbar_10 = [np.mean(np.random.exponential(scale=1/lambda_param, size=10)) for _ in range(Nsim)]
xbar_200 = [np.mean(np.random.exponential(scale=1/lambda_param, size=200)) for _ in range(Nsim)]

```

```

# Theoretical normal densities for CLT comparison
x_vals = np.linspace(0, 3, 400)
d10 = norm.pdf(x_vals, loc=true_mean, scale=np.sqrt(true_var / 10))
d200 = norm.pdf(x_vals, loc=true_mean, scale=np.sqrt(true_var / 200))

# Create side-by-side plots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Plot for n = 10
axes[0].hist(xbar_10, bins=40, density=True, color='grey', edgecolor='white', alpha=0.7)
axes[0].plot(x_vals, d10, color='blue', linewidth=2, label='Theoretical PDF')
axes[0].axvline(true_mean, color='red', linestyle='--', linewidth=1, label='True Mean')
axes[0].set_title('Sampling Distribution\nof xbar (n = 10)', fontsize=14)
axes[0].set_xlabel('xbar')
axes[0].set_ylabel('Density')
axes[0].legend()

# Plot for n = 200
axes[1].hist(xbar_200, bins=40, density=True, color='grey', edgecolor='white', alpha=0.7)
axes[1].plot(x_vals, d200, color='blue', linewidth=2, label='Theoretical PDF')
axes[1].axvline(true_mean, color='red', linestyle='--', linewidth=1, label='True Mean')
axes[1].set_title('Sampling Distribution\nof xbar (n = 200)', fontsize=14)
axes[1].set_xlabel('xbar')
axes[1].set_ylabel('Density')
axes[1].legend()

plt.tight_layout()
plt.show()

```

